MODUL PRAKTIKUM (2)

SISTEM BASIS DATA

2025

DAFTAR ISI

MODUL I	DATA DEFINITION LANGUAGE (DDL)	3
MODUL II	DATA MANIPULATION LANGUAGE (INSERT, UPDATE, DELETE DAN SELECT)	13
MODUL III	DATA MANIPULATION LANGUAGE (FUNGSI AGREGASI, GROUP BY DAN HAVING	21
MODUL IV	PENGAMBILAN DATA PADA LEBIH DARI SATU TABEL	27

MODUL I
DATA DEFINITION LANGUAGE (DDL)

Tujuan Pembelajaran :

1. Mahasiswa mengetahui perintah-perintah yang termasuk pada kategori Data Definition Language (DDL).
2. Mahasiswa mampu memahami perintah CREATE.
3. Mahasiswa mampu memahami perintah ALTER.
4. Mahasiswa mampu memahami perintah DROP.
5. Mahasiswa mampu memahami penggunaan Constraint.

Tes Awal :

1. Apa yang Anda ketahui tentang Data Definition Language (DDL)?
2. Apa saja perintah-perintah yang termasuk dalam kategori DDL?

DASAR TEORI

Data Definition Language atau disingkat DDL adalah bahasa khusus yang digunakan dalam merancang struktur basis data secara keseluruhan (Fathansyah, 2012). Di dalam kategori ini terdapat perintah-perintah sebagai berikut :

Tabel 1.1 Data Definition Language (DDL)

Perintah	Keterangan
CREATE DATABASE	Membuat Database
CREATE TABLE	Membuat tabel
ALTER TABLE	Mengubah struktur tabel seperti menambah field, mengubah field dan menghapus field.
DROP TABLE	Menghapus tabel dari database

Perintah CREATE

Sistem manajemen basis data (DBMS) memungkinkan untuk membuat dan mengelola banyak database independent. Untuk membuat database berikut perintahnya.

Sintaks 1.1:

```
CREATE DATABASE nama_database
```

Nama database tidak boleh mengandung spasi dan tidak boleh memiliki nama yang sama antar database.

Untuk dapat membuat tabel dalam database Anda perlu mengaktifkan terlebih dahulu database yang akan digunakan, untuk mengaktifkan database yang telah dibuat dapat menggunakan perintah **USE** nama_database . Untuk membuat tabel berikut querinya.

Sintaks 1.2:

```
CREATE TABLE nama_tabel
(nama_field_1 tipedata_1 [NOT NULL | NULL]
[{nama_field_2 tipedata_2 [NOT NULL | NULL]}…])
```

nama_tabel merupakan nama dari tabel yang akan di buat, sesuaikan nama_tabel dengan entitas yangdiwakilinya. **nama_field_1, nama_field_2** adalah nama field yang harus ada dalam dalam sebuah tabelyang mewakili atribut entitas tersebut. Tiap field akan menampung data dengan tipe data tertentu yang ditunjukkan oleh tipedata_1, tipedata_2 dan seterusnya. Opsi berikutnya adalah constraint (syarat/batasan) dari masing-masing tabel apakah boleh kosong (NULL) atau harus diisi (NOT NULL).

Tipe data merupakan suatu bentuk pemodelan data yang dideklarasikan pada saat melakukan pembuatan tabel. Tipe data akan berpengaruh pada setiap data yang akan dimasukkan ke dalam sebuah tabel. Data yang akan dimasukkan harus sesuai dengan tipe data yang dideklarasikan. Dalam MySQL terdapat beberapa tipe data, diantaranya:

Tabel 1.2 Tipe Data String

Type Data	Keterangan
CHAR	Mampu menangani data hingga 255 karakter.
VARCHAR	Mampu menangani data hingga 255 karakter. Tipe data VARCHAR tidakmengharuskan untuk memasukkan data sepanjang yang telah ditentukan.
TINYBLOB, TINYTEXT	Mampu menangani data sampai 2^8-1 data.
BLOB, TEXT	Type string yang mampu menangani data hingga 2^{16}-1.
MEDIUMBLOB, MEDIUMTEXT	Ukuran 16777215 byte. Mampu menyimpan data hingga 2^{24}-1.
LONGBLOB, LONGTEXT	Ukuran 4294967295 byte. Mampu menyimpan data hingga berukuran GIGABYTE. Tipe data ini memiliki batas penyimpanan hingga 2^{32}-1.

ENUM('nilai1','nil ai2',...,'nilaiN')	Ukuran 1 atau 2 byte. Tergantung jumlah nilai enumerasinya (maksimum65535 nilai)
SET('nilai1','nilai2 ',...,'nilaiN')	1,2,3,4 atau 8 byte, tergantung jumlah anggota himpunan (maksimum 64anggota)

Tabel 1.3 Tipe Data Numerik

Type Data	Keterangan
TINYINT	Bilangan bulat terkecil, dengan jangkauan untuk bilangan bertanda: -128 sampaidengan 127 dan untuk yang tidak bertanda : 0 s/d 255.
SMALLINT	Bilangan bulat dengan jangkauan untuk bilangan bertanda : -32768 s/d 32767 danuntuk yang tidak bertanda : 0 s/d 65535
MEDIUMINT	Bilangan bulat dengan jangkauan untuk bilangan bertanda : -8388608 s/ d 8388607dan untuk yang tidak bertanda : 0 s/d 16777215
INT	Bilangan bulat dengan jangkauan untuk bilangan bertanda : -2147483648 s/d 2147483647 dan untuk yang tidak bertanda : 0 s/d 4294967295
BIGINT	Bilangan bulat terbesar dengan jangkauan untuk bilangan bertanda : -9223372036854775808 s/d 9223372036854775807 dan untuk yang tidak bertanda : 0 s/d18446744737095516l5
FLOAT, REAL, DOUBLE	Bilangan pecahan
DECIMAL, NUMERIC	Bilangan pecahan, misalnya DECIMAL(5,2 dapat digunakan untukmenyimpan bilangan -99,99 s/d 99,99

Tabel 1.4 Tipe Data Date and Time

Type Data	Keterangan
DATETIME	Kombinasi tanggal dan jam, dengan jangkauan dari '1000-01-01 00:00:00' s/d'9999-12-31 23:59:59'
DATE	Tanggal dengan jangkauan dari '1000-01-01' s/d '9999-12-31'
TIMESTAMP	Kombinasi tanggal dan jam, dengan jangkauan dari '1970-01-01 00:00:01' s/d'2038-01-19 03:14:07
TIME	Waktu dengan jangkauan dari '-838:59:59' s/d '838:59:59'
YEAR	Data tahun antara 1901 s/d 2155

Constraint

Constaraint merupakan batasan atau aturan yang ada pada tabel. MySQL menyediakan beberapa tipe constaint berikut:

1. NOT NULL merupakan suatu kolom yang mendifinisikan dengan constraint NOT

NULL. Kolom yang berfungsi sebagai kunci primer (Primary Key) otomatis tidak boleh NULL.

2. UNIQUE mendifinisikan suatu kolom bersifat unik, artinya antara satu datadengan data lain namanya tidak boleh sama, misal alamat email.
3. PRIMARY KEY Constaint PRIMARY KEY membentuk key yang unik untuk suatu tabel.
4. FOREIGN KEY Constraint didefinisikan pada suatu kolom yang ada pada suatu tabel, dimana kolom tersebut juga dimiliki oleh tabel yang lain sebagai suatu PRIMARY KEY bisa digunakan untuk menghubungkan antara dua tabel.
5. CHECK constraint yang satu ini mendefinisikan sebuah kondisi untuk data agar dapat masuk dalam field artinya tiap pemasukan data atau editing terhadap data record, field yang dimasukkan akan selalu diperiksa apakah isinya ada diantara data-data yang dimasukkan, jika tidak ada maka SQL akan menampilkan pesan ERROR.
6. DEFAULT digunakan untuk mendefinisikan nilai default dari field yang mana ketika ada baris baru yang dimasukkan kedalam tabel nilai default dari field akan digunakan apabila tidak ada nilai yang diberikan padanya.

Perintah ALTER

Setelah membuat tabel dalam database, dapat memodifikasi field pada tabel yang telah dibuat. Dengan perintah ALTER dapat membuat perubahan pada struktur tabel tanpa menghapus dan menciptakan. Queri ALTER :

Sintaks 1.3 Queri untuk menambah field pada tabel:

```
ALTER TABLE nama_tabel ADD nama_field tipe_data (lebar)
```

Sintaks 1.4 Queri untuk merubah field pada tabel:

```
ALTER TABLE nama_tabel CHANGE COLUMN nama_field
tipe_data (lebar)
```

Sintaks 1.5 Queri untuk menghapus field pada tabel:

```
ALTER TABLE nama_tabel DROP COLUMN nama_field.
```

Perintah DROP

Perintah terakhir dari Data Definition Language, DROP memungkinkan untuk menghapus seluruh objek dalam database dari DBMS. Gunakan perintah ini dengan hati-hati, perlu diingat bahwa perintah DROP akan menghapus data keseluruhan struktur dari database. Querinya sebagai berikut.

Sintaks 1.6 Query untuk menghapus database:

```
DROP DATABASE nama_database
```

Sintaks 1.7 Query untuk menghapus Tabel:

```
DROP TABLE nama_tabel
```

KEGIATAN PRAKTIKUM

Untuk lebih memahami tentang DDL (Data Definition Language) maka buatlah database dengan nama "dbAkademik" yang mempunyai dua tabel yaitu tabel "Program Studi" dan tabel "Mahasiswa", isilahkedua tabel sesuai dengan ketentuan tabel dibawah ini!

Create database

Contoh 1.1

Membuat database baru dengan nama dbakademik

```
CREATE DATABASE dbakademik
```

Aktifkan database yang sudah dibuat

```
USE dbakademik
```

Create table

Contoh 1.2

Selanjutnya membuat tabel Program Studi dengan struktur tabel sebagai berikut:

Tabel 2.5 Tabel Program Studi

Nama Field	Tipe data	Lebar
Kode_Prodi	Char	3
Nama_Prodi	Varchar	50
Ketua_Prodi	Varchar	50

```
CREATE TABLE program_studi
(
kode_prodi CHAR(3) NOT NULL,
nama_prodi VARCHAR(50) NOT NULL,
ketua_prodi VARCHAR (50) NOT NULL,
PRIMARY KEY (kode_prodi)
)
```

Hasil pembuatan tabel program studi bisa dilihat menggunakan perintah:

DESC program_studi

Dengan perintah tersebut maka akan ditampilkan struktur tabel program studi seperti pada Gambar 1.1

```
+-------------+-------------+------+-----+---------+-------+
| Field       | Type        | Null | Key | Default | Extra |
+-------------+-------------+------+-----+---------+-------+
| kode_prodi  | char(3)     | NO   | PRI | NULL    |       |
| nama_prodi  | varchar(50) | NO   |     | NULL    |       |
| ketua_prodi | varchar(50) | NO   |     | NULL    |       |
+-------------+-------------+------+-----+---------+-------+
```

Gambar 1.1 Struktur tabel program_studi yang telah dibuat

Dari Gambar 1.1 dapat dilihat bahwa struktur tabel program_studi sesuai dengan perintah yang telah diberikan. Pada field kode_prodi bagian kolom Key terdapat keterangan "PRI" yang artinya field ini merupakan primary key dari tabel program_studi.

Contoh 1.3

Selanjutnya membuat tabel Mahasiswa dengan struktur tabel sebagai berikut:

Tabel 2.5 Tabel Mahasiswa

Nama Field	Tipe data	Lebar
NIM	Char	7
Nama_Mhs	Varchar	50
Alamat	Varchar	100
Kode_Prodi	Char	3

```
CREATE TABLE mahasiswa
(
nim CHAR(7) NOT NULL,
nama_mhs VARCHAR(50) NOT NULL,
alamat VARCHAR (100) NOT NULL,
kode_prodi  CHAR (3) NOT NULL,
PRIMARY KEY (nim)
)
```

Hasil pembuatan tabel mahasiswa bisa dilihat menggunakan perintah:

DESC mahasiswa

Dengan perintah tersebut maka akan ditampilkan struktur tabel mahasiswa seperti pada Gambar 1.2

```
+------------+-------------+------+-----+---------+-------+
| Field      | Type        | Null | Key | Default | Extra |
+------------+-------------+------+-----+---------+-------+
| nim        | char(7)     | NO   | PRI | NULL    |       |
| nama_mhs   | varchar(50) | NO   |     | NULL    |       |
| alamat     | varchar(50) | NO   |     | NULL    |       |
| kode_prodi | char(3)     | NO   |     | NULL    |       |
+------------+-------------+------+-----+---------+-------+
```

Gambar 1.2 Struktur tabel mahasiswa

ALTER

Berikutnya diberikan beberapa contoh penggunaan perintah **ALTER.**

Contoh 1.4

Menambahkan field “nomor_telepon” dan “jenis_kelamin” pada tabel mahasiswa.

```
ALTER TABLE mahasiswa
ADD (nomor_telepon VARCHAR(12) NOT NULL,
Jenis_kelamin VARCHAR(10) NOT NULL)
```

Hasil penambahan field “nomor_telepon” dan “jenis_kelamin” pada tabel mahasiswa dapat dilihat pada Gambar 1.3.

```
+---------------+-------------+------+-----+---------+-------+
| Field         | Type        | Null | Key | Default | Extra |
+---------------+-------------+------+-----+---------+-------+
| nim           | char(7)     | NO   | PRI | NULL    |       |
| nama_mhs      | varchar(50) | NO   |     | NULL    |       |
| alamat        | varchar(50) | NO   |     | NULL    |       |
| kode_prodi    | char(3)     | NO   |     | NULL    |       |
| nomor_telepon | varchar(12) | NO   |     | NULL    |       |
| jenis_kelamin | varchar(10) | NO   |     | NULL    |       |
+---------------+-------------+------+-----+---------+-------+
```

Gambar 1.3 Struktur tabel mahasiswa setelah menambahkan field nomor_telepon dan jenis_kelamin

Contoh 1.5

Mengubah field “nomor_telepon” dan “jenis_kelamin” pada tabel mahasiswa.

```
ALTER TABLE mahasiswa
CHANGE COLUMN jenis_kelamin jk VARCHAR (12) NOT NULL
```

```
+---------------+-------------+------+-----+---------+-------+
| Field         | Type        | Null | Key | Default | Extra |
+---------------+-------------+------+-----+---------+-------+
| nim           | char(7)     | NO   | PRI | NULL    |       |
| nama_mhs      | varchar(50) | NO   |     | NULL    |       |
| alamat        | varchar(50) | NO   |     | NULL    |       |
| kode_prodi    | char(3)     | NO   |     | NULL    |       |
| nomor_telepon | varchar(12) | NO   |     | NULL    |       |
| jk            | varchar(11) | NO   |     | NULL    |       |
+---------------+-------------+------+-----+---------+-------+
```

Gambar 1.4 Struktur tabel mahasiswa setelah mengubah field jenis_kelamin

Contoh 1.6

Menghapus field “jk” dan “nomor_telepon” pada tabel mahasiswa.

```
ALTER TABLE mahasiswa
DROP COLUMN jk,
DROP COLUMN nomor_telepon
```

Field	Type	Null	Key	Default	Extra
nim	char(7)	NO	PRI	NULL	
nama_mhs	varchar(50)	NO		NULL	
alamat	varchar(50)	NO		NULL	
kode_prodi	char(3)	NO		NULL	

Gambar 1.5 Struktur tabel mahasiswa setelah menghapus field jk dan nomor_telepon

Contoh 1.7

Perintah **ALTER** juga bisa digunakan untuk menambahkan constraint. Tabel mahasiswa perlu direlasikan dengan tabel program studi melalui field kode_prodi. Karena pada saat pembuatan tabel mahasiswa diawal tadi belum menetapkan foreign key maka untuk menjadikan field kode_prodi sebagai foreign key pada tabel mahasiswa, bisa menggunakan perintah **ALTER** seperti query berikut ini.

```
ALTER TABLE mahasiswa
ADD CONSTRAINT
fk_mahasiswa_relation_prodi FOREIGN
KEY (kode_prodi)
REFERENCES program_studi (kode_prodi)
```

Perubahan struktur tabel mahasiswa karena adanya perintah **ALTER** yang baru saja dilakukan dapat dilihat menggunakan perintah:

DESC mahasiswa

Dengan perintah tersebut maka akan ditampilkan struktur tabel mahasiswa seperti pada Gambar 1.6

Field	Type	Null	Key	Default	Extra
nim	char(7)	NO	PRI	NULL	
nama_mhs	varchar(50)	NO		NULL	
alamat	varchar(50)	NO		NULL	
kode_prodi	char(3)	NO	MUL	NULL	
nomor_telepon	varchar(12)	NO		NULL	

Gambar 1.6 Struktur tabel mahasiswa setelah ditetapkan foreign key

Dapat dilihat pada Gambar 1.3 bahwa pada field kode_prodi di bagian key tertera kode "MUL" yang artinya field ini merupkan kunci yang tidak unik. Dengan adanya kode ini pada field kode_prodi itu berarti bahwa perintah **ALTER** yang dilakukan untuk menetapkan kode_prodi sebagai foreign key pada tabel mahasiswa berhasil dieksekusi.

DROP

Contoh 1.8

Berikutnya diberikan contoh penggunaan perintah **DROP**. Misalkan ingin menghapus tabel mahasiswa yang sudah dibuat maka perintah yang bisa digunakan adalah sebagai berikut.

```
DROP TABLE mahasiswa
```

LATIHAN 1

1. Buatlah database baru dengan perintah SQL dengan nama "dbtoko" yang memiliki 2 tabel yaitu tabel "barang" dan tabel "pembelian" dengan ketentuan sebagai berikut:
   a. Tabel barang memiliki field: id_barang, nama_barang, harga, stok
      Tabel pembelian memiliki field: id_pembeli, id_barang, tgl_beli, jumlah_beli
   b. Tentukan tipe data masing-masing field
2. Modifikasi tabel tersebut dengan menambahkan primary key serta foreign Key nya agar kedua tabel saling berelasi.

MODUL II
DATA MANIPULATION LANGUAGE
(INSERT, UPDATE, DELETE DAN SELECT)

Tujuan Pembelajaran:

1. Mahasiswa bisa melakukan manipulasi data pada basis data.
2. Mahasiswa bisa memahami perintah INSERT
3. Mahasiswa bisa memahami perintah UPDATE
4. Mahasiswa bisa memahami perintah DELETE
5. Mahasiswa bisa memahami perintah SELECT

Tes Awal :

1. Apa yang Anda ketahui tentang Data Manipulation Language (DML)?
2. Jelaskan perbedaan antara DDL dan DML!
3. Apa yang Anda ketahui tentang perintah INSERT, UPDATE, dan DELETE?
4. Sebutkan macam-macam klausa dan operator pada perintah SELECT!

DASAR TEORI

Data Manipulation Language (DML) merupakan kelompok bahasa basis data yang bisa digunakan pengguna untuk mengakses atau memanipulasi data yang berbentuk suatu model data tertentu (Silberschatz, 2011). Perintah-perintah umum yang termasuk dalam kategori DML yang disediakan dalam MySQL adalah sebagai berikut:

1. **INSERT**, merupakan perintah yang dapat digunakan untuk menambahkan data (*tuple*) baru ke dalam suatu tabel.
2. **UPDATE**, merupakan perintah yang dapat digunakan untuk mengubah data yang yang sudah tersimpan sebelumnya.
3. **DELETE**, merupakan perintah untuk menghapus data dalam suatu tabel.
4. **SELECT**, merupakan perintah yang digunakan untuk mengambil atau menampilkan data dari tabel. Perintah ini adalah perintah dasar yang digunakan untuk mengambil informasi dari basis data

Untuk dapat lebih memahami penggunaan perintah-perintah DML, berikut ini adalah penjelasan dalam bentuk kegiatan praktikum untuk masing-masing dari perintah-perintah di atas.

KEGIATAN PRAKTIKUM

INSERT

Untuk dapat melakukan kegiatan praktikum, diperlukan struktur basis data yang sudah ada sebelumnya. Oleh karena itu, stuktur basis data yang digunakan disini adalah struktur basis data yang dibuat dalam MODUL I yaitu dbakademik. Setelah tabel dibuat beserta constraint-constraint yang diperlukan, maka tabel siap digunakan untuk menampung data. Perintah INSERT merupakan perintah yang digunakan untuk memasukkan data ke dalam tabel. Bentuk umum dari perintah INSERT adalah sebagai berikut :

Sintaks 2.1

```
INSERT INTO nama_tabel
      (field1,field2,field3,..)
      VALUES (value1,value2,value3,..)
```

Catatan:

Bentuk sintaks di atas digunakan jika ingin memasukkan data pada kolom tertentu saja dengan syarat jumlah field = jumlah nilai.

Untuk memasukkan data pada semua field yang ada dalam suatu tabel maka bentuk sintaksnya adalah sebagai berikut:

Sintaks 2.2

```
INSERT INTO nama_tabel
      VALUES (value1,value2,value3,..)
```

Catatan:

Nilai-nilai dimasukkan mulai dari field pertama, sehingga harus memperhatikan urutan field yang ada di dalam tabel agar nilai yang dimasukkan sesuai dengan field yang seharusnya.

Contoh 2.1

Memasukkan data pada tabel program_studi di database dbakademik.

Menggunakan sintaks 2.1:

```
INSERT INTO program_studi
      (kode_prodi,nama_prodi,ketua_prodi)
      VALUES('PS1','Sistem Informasi','Yufika Sari Bagi');
```

Atau menggunakan sintaks 2.2:

```
INSERT INTO program_studi
      VALUES('PS1','Sistem Informasi','Yufika Sari Bagi');
```

Untuk memasukkan beberapa data sekaligus, dapat menggunakan perintah sebagai berikut:

```
INSERT INTO program_studi
      VALUES('PS1','Sistem Informasi', 'Yufika Sari Bagi'),
            ('PS2','Teknik Informatika', 'Michel Farrel Tomatala');
```

Selanjutnya masukkan data pada Tabel 2.1 berikut ini ke dalam tabel mahasiswa pada database dbakademik.

Tabel 2.1 Data Mahasiswa

NIM	Nama Mahasiswa	Alamat	Program Studi
1517108	Harioko Mamonto	Modayag	Sistem Informasi
1517119	Stivania Wuwung	Bongkudai	Sistem Informasi
1517211	Rida Sanggilalung	Mopait	Teknik Informatika
1517121	Rinilita Kalengkongan	Kotamobagu	Sistem Informasi
1517220	Farhan Monigi	Kotamobagu	Teknik Informatika
1517208	Marselino Mogot	Kotamobagu	Teknik Informatika
1517112	Miliano Rindengan	Kotobangon	Teknik Informatika
1517104	Angie Enji Tombeng	Kotobangon	Sistem Informasi
1517221	Fachri Gobel	Kotamobagu	Teknik Informatika
1315111	Utami Mohune	Kotamobagu	Sistem Informasi

Catatan:

Untuk program studi mahasiswa yang disimpan yaitu kode program studi dari setiap mahasiswa.

SELECT

Select merupakan perintah yang digunakan untuk mengambil atau menampilkan data dari tabel. Untuk bisa melihat data yang sudah dimasukkan pada contoh 2.1 menggunakan perintah **SELECT**. Bentuk-bentuk sintaks **SELECT** adalah sebagai berikut:

1. Menampilkan data keseluruhan

```
SELECT * FROM nama_tabel
```

2. Menampilkan semua data dengan seleksi kolom tertentu

```
SELECT nama_kolom_1, nama_kolom_2 FROM nama_tabel
```

3. Menampilkan data dengan kondisi tertentu

```
SELECT nama_kolom_1, nama_kolom_2 FROM nama_tabel
WHERE kondisi
```

Contoh 2.2

Untuk melihat semua data pada tabel program_studi gunakan perintah:

```
SELECT * FROM program_studi
```

Simbol * berarti menampilkan semua kolom yang ada di tabel program studi. Jika ingin menampilkan kolom tertentu saja maka jangan menggunakan simbol * tetapi langsung tuliskan kolom apa yang ingin ditampilkan.

Berikut contoh untuk melihat data pada tabel program_studi khusus kolom kode program studi dan ketua program studi saja.

```
SELECT kode_prodi, ketua_prodi FROM program_studi
```

kode_prodi	nama_prodi	ketua_prodi
PS1	Sistem Informasi	Yufika Sari Bagi
PS2	Teknik Informatika	Michel Farrel Tomatala

(a)

kode_prodi	ketua_prodi
PS1	Yufika Sari Bagi
PS2	Michel Farrel Tomatala

(b)

Gambar 2.1 (a) Tampilan keseluruhan data program studi, (b) Tampilan data program studi dengan kolom tertentu

Untuk melihat semua data pada tabel mahasiswa gunakan perintah:

```
SELECT * FROM mahasiswa
```

Untuk melihat data mahasiswa khusus kolom nama mahasiswa dan kode program studi:

```
SELECT nama_mhs, kode_prodi FROM mahasiswa
```

nim	nama_mhs	alamat	kode_prodi
1315111	Utami Mohune	Kotamobagu	PS1
1517104	Angie Enji Tombeng	Kotobangon	PS1
1517108	Harioko Mamonto	Modayag	PS1
1517112	Miliano Rindengan	Kotobangon	PS1
1517119	Stivania Wuwung	Bongkudai	PS1
1517121	Rinilita Kalengkongan	Kotamobagu	PS1
1517208	Marselino Mogot	Kotamobagu	PS2
1517211	Rida Sanggilalung	Mopait	PS2
1517220	Farhan Monigi	Kotamobagu	PS2
1517221	Fachri Gobel	Kotamobagu	PS2

(a)

nama_mhs	kode_prodi
Utami Mohune	PS1
Angie Enji Tombeng	PS1
Harioko Mamonto	PS1
Miliano Rindengan	PS1
Stivania Wuwung	PS1
Rinilita Kalengkongan	PS1
Marselino Mogot	PS2
Rida Sanggilalung	PS2
Farhan Monigi	PS2
Fachri Gobel	PS2

(b)

Gambar 2.2 (a) Tampilan keseluruhan data mahasiswa, (b) Tampilan data mahasiswa dengan kolom tertentu

Contoh 2.3

Berikutnya contoh penggunaan perintah select untuk menampilkan data mahasiswa dengan kondisi tertentu.

1. Misalkan ingin menampilkan data mahasiswa khusus yang beralamat di Kotamobagu, maka perintahnya yaitu:

```
SELECT * FROM mahasiswa WHERE alamat= 'Kotamobagu'
```

nim	nama_mhs	alamat	kode_prodi
1315111	Utami Mohune	Kotamobagu	PS1
1517121	Rinilita Kalengkongan	Kotamobagu	PS1
1517208	Marselino Mogot	Kotamobagu	PS2
1517220	Farhan Monigi	Kotamobagu	PS2
1517221	Fachri Gobel	Kotamobagu	PS2

Gambar 2.3 Tampilan data mahasiswa dengan kondisi alamat Kotamobagu

2. Misalkan ingin menampilkan data mahasiswa khusus yang beralamat di Kotamobagu dan prodi Sistem Informasi maka perintahnya yaitu:

```
SELECT * FROM mahasiswa
WHERE alamat= 'Kotamobagu' and kode_prodi= 'PS1'
```

nim	nama_mhs	alamat	kode_prodi
1315111	Utami Mohune	Kotamobagu	PS1
1517121	Rinilita Kalengkongan	Kotamobagu	PS1

Gambar 2.4 Tampilan data mahasiswa dengan kondisi alamat Kotamobagu dan prodi Sistem Informasi

UPDATE

Perintah **UPDATE** digunakan untuk mengubah data pada tabel. Bentuk umum sintaks UPDATE adalah sebagai berikut :

```
UPDATE nama_tabel
      SET nama_field = 'nilai_baru'
      WHERE kondisi
```

Contoh 2.4

Misalkan pada tabel mahasiswa, kita ingin mengubah alamat dari mahasiswa a.n. Utami Mohune dengan nim 1315111 yang sebelumnya alamatnya Kotamobagu menjadi Kotobangon, maka perintah sebagai berikut:

```
UPDATE mahasiswa
      SET alamat = 'Kotobangon'
      WHERE nim = '1315111'
```

Pada perintah di atas, kondisi yang digunakan yaitu nim karena nim tidak akan ada yang sama, sedangkan nama ada kemungkinan nama Utami Mohune yang sama, sehingga untuk menjaga agar tidak terjadi kesalahan data, maka kita perlu memperhatikan bagian kondisi ketika ingin melakukan update data.

nim	nama_mhs	alamat	kode_prodi
1315111	Utami Mohune	Kotobangon	PS1
1517104	Angie Enji Tombeng	Kotobangon	PS1
1517108	Harioko Mamonto	Modayag	PS1
1517112	Miliano Rindengan	Kotobangon	PS1
1517119	Stivania Wuwung	Bongkudai	PS1
1517121	Rinilita Kalengkongan	Kotamobagu	PS1
1517208	Marselino Mogot	Kotamobagu	PS2
1517211	Rida Sanggilalung	Mopait	PS2
1517220	Farhan Monigi	Kotamobagu	PS2
1517221	Fachri Gobel	Kotamobagu	PS2

Gambar 2.5 Tampilan data mahasiswa setelah update alamat dari mahasiswa dengan nim 1315111

DELETE

Perintah **DELETE** digunakan untuk menghapus data pada tabel. Bentuk umum sintaks DELETE adalah sebagai berikut :

1. Menghapus seluruh baris data/record yang ada di dalam tabel

```
DELETE FROM nama_tabel
```

2. Menghapus baris data/record berdasarkan suatu kondisi tertentu

```
DELETE FROM nama_tabel
      WHERE kondisi
```

Contoh 2.5

Misalkan pada tabel mahasiswa, kita ingin menghapus record dengan nim 1315111, maka perintahnya sebagai berikut:

```
DELETE FROM mahasiswa
      WHERE nim = '1315111'
```

nim	nama_mhs	alamat	kode_prodi
1517104	Angie Enji Tombeng	Kotobangon	PS1
1517108	Harioko Mamonto	Modayag	PS1
1517112	Miliano Rindengan	Kotobangon	PS1
1517119	Stivania Wuwung	Bongkudai	PS1
1517121	Rinilita Kalengkongan	Kotamobagu	PS1
1517208	Marselino Mogot	Kotamobagu	PS2
1517211	Rida Sanggilalung	Mopait	PS2
1517220	Farhan Monigi	Kotamobagu	PS2
1517221	Fachri Gobel	Kotamobagu	PS2

Gambar 2.6 Tampilan data mahasiswa setelah menggunakan perintah delete

LATIHAN 2

Praktekkan masing-masing perintah INSERT, SELECT, UPDATE dan DELETE pada tabel barang dan pembelian yang sudah Anda buat pada Latihan 1. Data ditentukan sendiri.

MODUL III

DATA MANIPULATION LANGUAGE

(FUNGSI AGREGASI, GROUP BY DAN HAVING)

Tujuan Pembelajaran:

1. Mahasiswa mengetahui dan memahami perintah fungsi Agregasi, Group By dan Having
2. Mahasiswa bisa menggunakan perintah fungsi Agregasi, Group By dan Having

Tes Awal :

Apa yang Anda ketahui tentang fungsi SUM, AVG, COUNT, MIN dan MAX?

DASAR TEORI

Fungsi agregasi merupakan fungsi yang digunakan untuk mengambil suatu nilai dari sekumpulan data atau beberapa kumpulan data yang telah dikelompokkan (Silberschatz, 2011). Secara umum fungsi-fungsi agregasi yang tersedia pada SQL adalah sebagai berikut:

1. **SUM**, merupakan fungsi yang bisa digunakan untuk menghitung total nilai.
2. **AVG**, merupakan fungsi yang bisa digunakan untuk menghitung rata-rata nilai.
3. **MIN**, merupakan fungsi yang bisa digunakan untuk mendapatkan nilai minimal/terendah dari suatu kolom.
4. **MAX**, merupakan fungsi yang bisa digunakan untuk mendapatkan nilai maksimal/tertinggi dari suatu kolom.
5. **COUNT,** merupakan fungsi yang bisa digunakan untuk menghitung banyaknya baris yang dipilih secara query.

Untuk dapat lebih memahami penggunaan fungsi-fungsi agregasi, berikut ini adalah penjelasan dalam bentuk kegiatan praktikum.

KEGIATAN PRAKTIKUM

FUNGSI AGREGASI DASAR

Berikut bentuk sintaks dari fungsi agregasi SUM, AVG, MIN, MAX dan COUNT.

Fungsi SUM

```
SELECT SUM(kolom)
FROM nama_tabel
```

Fungsi AVG

```
SELECT AVG(kolom)
FROM nama_tabel
```

Fungsi MIN

```
SELECT MIN(kolom)
FROM nama_tabel
```

Fungsi MAX

```
SELECT MAX(kolom)
FROM nama_tabel
```

Fungsi COUNT

```
SELECT COUNT(kolom)
FROM nama_tabel
```

Sebelum lanjut ke contoh penggunaan fungsi agregasi, maka terlebih dahulu tambahkan field "total_sks" pada tabel mahasiswa dengan tipe data integer (INT) dengan lebar 3, sehingga struktur tabel mahasiswa menjadi seperti pada Gambar 3.1.

```
+------------+-------------+------+-----+---------+-------+
| Field      | Type        | Null | Key | Default | Extra |
+------------+-------------+------+-----+---------+-------+
| nim        | char(7)     | NO   | PRI | NULL    |       |
| nama_mhs   | varchar(50) | NO   |     | NULL    |       |
| alamat     | varchar(50) | NO   |     | NULL    |       |
| kode_prodi | char(3)     | NO   | MUL | NULL    |       |
| total_sks  | int(3)      | YES  |     | NULL    |       |
+------------+-------------+------+-----+---------+-------+
```

Gambar 3.1 Struktur tabel mahasiswa setelah menambahkan field "total_sks"

Kemudian lakukan update terhadap data yang sudah ada sesuai dengan Tabel 3.1.

Tabel 3.1 Data Mahasiswa

NIM	Nama Mahasiswa	Total SKS yang sudah dicapai
1517108	Harioko Mamonto	140
1517119	Stivania Wuwung	135
1517211	Rida Sanggilalung	144
1517121	Rinilita Kalengkongan	110
1517220	Farhan Monigi	140
1517208	Marselino Mogot	130
1517112	Miliano Rindengan	140
1517104	Angie Enji Tombeng	140
1517221	Fachri Gobel	144

Tampilan data pada tabel mahasiswa setelah melakukan update total sks sesuai pada Tabel 3.1 ditunjukkan pada Gambar 3.2.

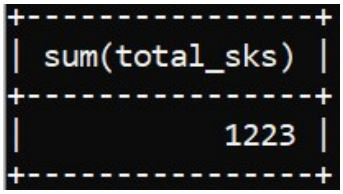
nim	nama_mhs	alamat	kode_prodi	total_sks
1517104	Angie Enji Tombeng	Kotobangon	PS1	140
1517108	Harioko Mamonto	Modayag	PS1	140
1517112	Miliano Rindengan	Kotobangon	PS1	140
1517119	Stivania Wuwung	Bongkudai	PS1	135
1517121	Rinilita Kalengkongan	Kotamobagu	PS1	110
1517208	Marselino Mogot	Kotamobagu	PS2	130
1517211	Rida Sanggilalung	Mopait	PS2	144
1517220	Farhan Monigi	Kotamobagu	PS2	140
1517221	Fachri Gobel	Kotamobagu	PS2	144

Gambar 3.2 Tampilan data mahasiswa setelah melakukan update data

Contoh 3.1

1. Menampilkan total nilai dari kolom total_sks bisa menggunakan fungsi SUM.

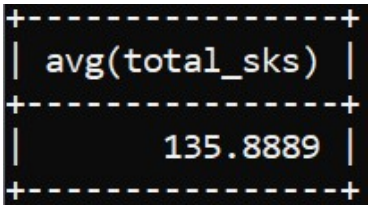
```
SELECT SUM(total_sks)
FROM mahasiswa
```



Gambar 3.3 Hasil fungsi SUM kolom total_sks pada tabel mahasiswa

2. Menampilkan rata-rata nilai dari kolom total_sks bisa menggunakan fungsi SUM.

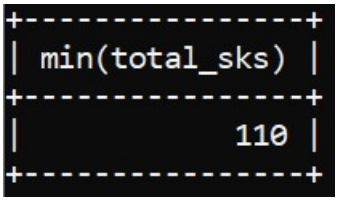
```
SELECT AVG(total_sks)
FROM mahasiswa
```



Gambar 3.4 Hasil fungsi AVG kolom total_sks pada tabel mahasiswa

3. Menampilkan total sks yang paling rendah

```
SELECT MIN(total_sks)
FROM mahasiswa
```



Gambar 3.5 Hasil fungsi MIN kolom total_sks pada tabel mahasiswa

4. Menampilkan total sks yang paling banyak

```
SELECT MAX(total_sks)
FROM mahasiswa
```

```
+---------------+
| max(total_sks) |
+---------------+
|           144 |
+---------------+
```

Gambar 3.6 Hasil fungsi MAX kolom total_sks pada tabel mahasiswa

5. Menampilkan banyaknya data mahasiswa yang ada di tabel mahasiswa

```
SELECT COUNT(*)
FROM mahasiswa
```

```
+----------+
| count(*) |
+----------+
|        9 |
+----------+
```

Gambar 3.7 Hasil fungsi COUNT pada tabel mahasiswa

FUNGSI AGREGASI – GROUP BY

Fungsi agregasi dengan group by digunakan untuk membentuk kelompok/group dari tuple yang memiliki nilai yang sama.

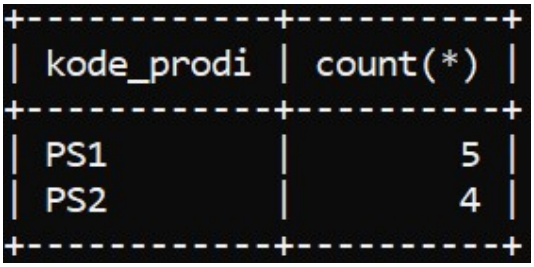
Sintaks:

```
SELECT FungsiAgregasi(kolom)
FROM nama_tabel
GROUP BY kolom
```

Contoh 3.2

Menampilkan banyaknya mahasiswa setiap program studi.

```
SELECT kode_prodi, COUNT(*)
FROM mahasiswa
GROUP BY kode_prodi
```



kode_prodi	count(*)
PS1	5
PS2	4

Gambar 3.8 Hasil fungsi COUNT pada tabel mahasiswa dikelompokkan berdasarkan kode_prodi

FUNGSI AGREGASI – HAVING

Having merupakan suatu klausa yang digunakan untuk membatasi baris yang dihasilkan oleh fungsi agregasi group by.

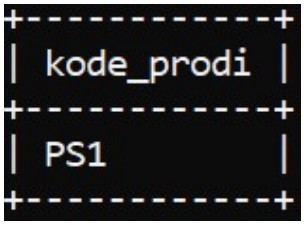
Sintaks:

```
SELECT FungsiAgregasi(kolom)
FROM nama_tabel
GROUP BY kolom
HAVING kondisi
```

Contoh 3.3

Menampilkan kode prodi yang banyak mahasiswanya lebih dari 4.

```
SELECT kode_prodi
FROM mahasiswa
GROUP BY kode_prodi
HAVING COUNT(*)>4
```



kode_prodi
PS1

Gambar 3.8 Hasil fungsi COUNT pada tabel mahasiswa dikelompokkan berdasarkan kode_prodi dengan suatu kondisi

MODUL IV

PENGAMBILAN DATA PADA LEBIH DARI SATU TABEL

Tujuan Pembelajaran:

1. Mahasiswa mengetahui perintah-perintah SQL untuk pengambilan data dari banyak tabel.
2. Mahasiswa mengetahui dan memahami macam-macam join.

Tes Awal :

1. Apa yang anda ketahui tentang JOIN?
2. Sebutkan macam-macam JOIN !
3. Apakah perbedaan Left JOIN dengan Right JOIN?

DASAR TEORI

Tipe-tipe JOIN

Terkadang dalam mengakses database, diperlukan data dari beberapa tabel yang saling berhubungan. Perintah JOIN dapat digunakan untuk mendapatkan data dari beberapa tabel (Alam, 2005). Kegunaan dari JOIN yaitu untuk menggabungkan data dari dua buah tabel atau lebih. Perintah JOIN dapat digunakan pada tabel biasa ataupun suatu VIEW. Macam-macam JOIN yaitu:

- INNER JOIN
- LEFT JOIN (LEFT OUTER JOIN)
- RIGHT JOIN (RIGHT OUTER JOIN)

Untuk dapat lebih memahami penggunaan perintah JOIN, berikut ini adalah penjelasan dalam bentuk kegiatan praktikum.

KEGIATAN PRAKTIKUM

INNER JOIN

Perintah JOIN yang paling umum yaitu INNER JOIN yang bisa digunakan pada semua basis data. Penggunaan perintah ini hanya bisa dilakukan pada dua buah tabel yang memiliki relasi. Data pada tabel pertama akan dihubungkan dengan data pada tabel kedua apabila nilai *field* yang dijadikan patokan relasi kedua tabel memiliki nilai yang sama maka data tersebut diambil. Bentuk umum perintah INNER JOIN :

```
SELECT field_1, field_2, field_n
FROM tabel_1
INNER JOIN tabel_2
ON tabel_1.field_key = tabel_2.field_key
```

Contoh 4.1

Pada database dbakademik yang telah dibuat sebelumnya terdapat tabel program_studi dan tabel mahasiswa dimana kedua tabel ini memiliki relasi lewat kode_prodi. Pada contoh kali ini perintah INNER JOIN akan digunakan pada kedua tabel tersebut untuk menampilkan nama mahasiswa dan nama program studi.

```
SELECT mahasiswa.nama_mhs, program_studi.nama_prodi
FROM mahasiswa
INNER JOIN program_studi
ON mahasiswa.kode_prodi = program_studi.kode_prodi
```

```
+----------------------+--------------------+
| nama_mhs             | nama_prodi         |
+----------------------+--------------------+
| Angie Enji Tombeng   | Sistem Informasi   |
| Harioko Mamonto      | Sistem Informasi   |
| Miliano Rindengan    | Sistem Informasi   |
| Stivania Wuwung      | Sistem Informasi   |
| Rinilita Kalengkongan| Sistem Informasi   |
| Marselino Mogot      | Teknik Informatika |
| Rida Sanggilalung    | Teknik Informatika |
| Farhan Monigi        | Teknik Informatika |
| Fachri Gobel         | Teknik Informatika |
+----------------------+--------------------+
```

Gambar 4.1 Hasil INNER JOIN tabel mahasiswa dan tabel program studi

LEFT JOIN

Perintah LEFT JOIN (*left outer join*) akan menampilkan semua baris data pada tabel kiri walaupun tidak memiliki relasi pada tabel di bagian kanan. Data yang tidak berelasi akan diberi nilai NULL. Bentuk perintahnya sebagai berikut:

```
SELECT field_1, field_2, field_n
FROM tabel_1
LEFT JOIN tabel_2
ON tabel_1.field_key = tabel_2.field_key
```

Contoh 4.2

Sebelum lanjut ke contoh penggunaan LEFT JOIN, silahkan tambah 1 record baru pada tabel program_studi misalnya kode prodi=PS3, nama prodi=Komputer Akuntansi, ketua prodi=Mihuandayani agar dapat dilihat perbedaan LEFT JOIN dan INNER JOIN. Selanjutnya gunakan LEFT JOIN antara tabel program studi dengan mahasiswa.

```
SELECT program_studi.kode_prodi,
program_studi.nama_prodi, mahasiswa.nim,
mahasiswa.nama_mhs
FROM program_studi
LEFT JOIN mahasiswa
ON program_studi.kode_prodi = mahasiswa.kode_prodi
```

kode_prodi	nama_prodi	nim	nama_mhs
PS1	Sistem Informasi	1517104	Angie Enji Tombeng
PS1	Sistem Informasi	1517108	Harioko Mamonto
PS1	Sistem Informasi	1517112	Miliano Rindengan
PS1	Sistem Informasi	1517119	Stivania Wuwung
PS1	Sistem Informasi	1517121	Rinilita Kalengkongan
PS2	Teknik Informatika	1517208	Marselino Mogot
PS2	Teknik Informatika	1517211	Rida Sanggilalung
PS2	Teknik Informatika	1517220	Farhan Monigi
PS2	Teknik Informatika	1517221	Fachri Gobel
PS3	Komputer Akuntansi	NULL	NULL

Gambar 4.2 Hasil LEFT JOIN tabel program studi dan mahasiswa

RIGHT JOIN

Perintah RIGHT JOIN (*right outer join*) merupakan kebalikan dari LEFT JOIN yaitu akan menampilkan semua data pada tabel kanan walaupun tidak memiliki relasi pada tabel di bagian kiri. Data yang tidak berelasi akan diberi nilai NULL. Bentuk perintahnya sebagai berikut:

```
SELECT field_1, field_2, field_n
FROM tabel_1
RIGHT JOIN tabel_2
ON tabel_1.field_key = tabel_2.field_key
```

Contoh 4.2

Pada contoh kali ini gunakan RIGHT JOIN antara tabel program studi dengan mahasiswa.

```
SELECT program_studi.kode_prodi,
program_studi.nama_prodi, mahasiswa.nim,
mahasiswa.nama_mhs
FROM program_studi
RIGHT JOIN mahasiswa
ON program_studi.kode_prodi = mahasiswa.kode_prodi
```

kode_prodi	nama_prodi	nim	nama_mhs
PS1	Sistem Informasi	1517104	Angie Enji Tombeng
PS1	Sistem Informasi	1517108	Harioko Mamonto
PS1	Sistem Informasi	1517112	Miliano Rindengan
PS1	Sistem Informasi	1517119	Stivania Wuwung
PS1	Sistem Informasi	1517121	Rinilita Kalengkongan
PS2	Teknik Informatika	1517208	Marselino Mogot
PS2	Teknik Informatika	1517211	Rida Sanggilalung
PS2	Teknik Informatika	1517220	Farhan Monigi
PS2	Teknik Informatika	1517221	Fachri Gobel

Gambar 4.3 Hasil RIGHT JOIN tabel program studi dan mahasiswa